J - Jump to label.

Syntax:

```
J(label)
J(label)T
J(label)F
```

Description:

The Jump to label commands are similar to GOTO statements in other languages. When FRED sees

```
j(label)
```

FRED scans the buffer being executed until it finds a matching

```
@(label)
```

Execution then continues with the command following the label.

The search for a label begins at the `J(label)` command, goes to the end of the buffer that contains the command, and wraps around to the top if necessary. Stream directives such as `\B` and `\N` are **not** expanded once FRED has begun its search.

The syntax rules for labels are similar to those for buffer names. A label can be up to 16 characters long and must be enclosed in parentheses unless the label is a single character and the `O-I(` option is in effect. Any character may be used in a label name with the exception of `@` and `)`.

Stream directives are accepted and expanded in the labels for the `J` command. Thus

```
n(xx):2 j(label\n(xx))
```

will jump to `@(label2)`. This feature can be used to create the equivalent of computed GOTO's.

If a `J(label)` command is found in a `G` or `U` command list, FRED searches for `@(label)` in the hidden buffer used by those commands. If this label is not found, FRED issues an error.

`J(label)` jumps unconditionally to the specified label. If there are commands that follow the command on the same line, there must be a space immediately after the `J(label)`.

```
J(label)T
```

jumps only if the condition register is TRUE, while

```
J(label)F
```

jumps only if the condition register is FALSE. No space is necessary between these two commands and other commands on the line.

All Jump to label commands must be executed from inside a buffer. You will receive an error you type this kind of Jump command directly from the terminal. However, these `J` commands may be used in `G` or `U` command lists.